M2: Introduction to R Markdown and Reproducible Reports

Environmental Data Exploration

August 2026

Welcome

This file is a small **sandbox** for learning the basic structure of an R Markdown document. In the video, we will use it to see how written explanation, R code, results, and figures can live together in one reproducible source file.

An R Markdown document has three main components:

1. **YAML** – document-level information and settings at the top of the file.
2. **Markdown text** – the narrative that explains the analysis.
3. **R code chunks** – executable code that performs calculations and produces results.

Markdown text

Markdown lets us add simple formatting without using a traditional word processor.

Headings

Headings organize a document into meaningful sections. One `#` creates a major heading, while additional `#` symbols create lower-level headings.

Emphasis

We can write *italic text* or **bold text**.

Lists

A numbered list is useful when sequence matters:

1. Organize the project.
2. Write and run the analysis.
3. Interpret the results.
4. Communicate the results.

A bulleted list is useful when order does not matter:

- data

- code
- figures
- tables

R code chunks

Regular Markdown text is written for the reader. R code belongs inside an R code chunk so that R can execute it.

The chunk below performs a simple calculation.

```
5 + 2 * 3
```

```
## [1] 11
```

Try changing the expression and running the chunk again.

Code, results, and narrative together

One advantage of R Markdown is that the explanation and the analysis can remain connected.

Suppose we have five environmental temperature observations:

```
temperature <- c(18, 20, 19, 22, 35)
temperature
```

```
## [1] 18 20 19 22 35
```

We can calculate their mean using R:

```
mean(temperature)
```

```
## [1] 22.8
```

Because the calculation is part of the document, it can be reproduced whenever the document is run again.

Inline R code

R can also insert a calculated result directly into a sentence. For example:

The mean temperature in this small example is **22.8 degrees**.

If the values stored in `temperature` change, the value in that sentence will also change the next time the document is knitted.

A simple figure

R Markdown can also include figures generated by R code.

```r
plot(
  temperature,
  type = "b",
  xlab = "Observation",
  ylab = "Temperature",
  main = "Example temperature observations"
)
```

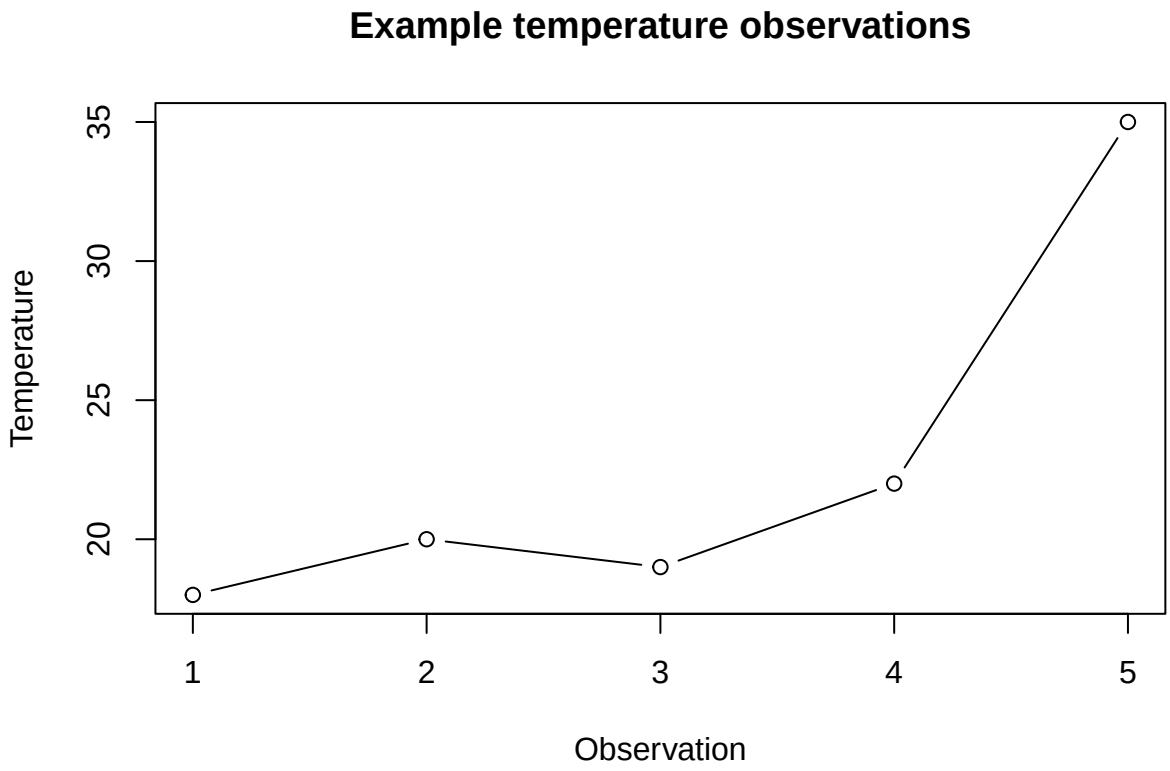

Figure 1: Temperature observations in the introductory example.

Later in the module, we will learn how to control figure size, captions, code visibility, and other report settings.

Knit the document

When you **Knit** this `.Rmd` file, RStudio processes the Markdown and executes the R code to create a finished document.

For now, focus on the basic workflow:

write -> code -> knit -> review

Try knitting this document and compare the source `.Rmd` file with the finished output.

Analysis document versus final report

While developing an analysis, an R Markdown file may contain code, intermediate results, notes, and diagnostic information that are useful to the analyst.

A final report is designed for an audience and may show only the information needed to understand the analysis and its results.

Later in this module, we will learn how to use YAML settings, code-chunk options, Markdown formatting, figures, tables, and inline R to turn a working analysis into a clear and reproducible report.

Try it yourself

Before moving on, experiment with this file:

1. Change one of the temperature values and rerun the relevant chunks.
2. Notice how the calculated mean changes.
3. Knit the document again and check whether the inline result updates.
4. Add a new Markdown heading.
5. Add one sentence in **bold** and one in *italics*.
6. Insert a new R code chunk and perform a simple calculation.

The goal is not to memorize the syntax. The goal is to become comfortable with the relationship between **source text, executable code, and the knitted report**.